Software Engineering Lab 4

Name – Neha Rastogi

SRN – PES1UG24AM175

Section – C

ChatGPT link: https://chatgpt.com/s/t_6ad0f413af648191b74d20a122b9e4a2

ChatGPT said:

Download the integrated code

I prepared a complete replacement project that includes **Tasks 1–4 together**, rather than treating each task as an isolated change.

Download the updated Brick Breaker project (ZIP) ↗

The ZIP contains:

- `main.py` — updated game loop and replay handling.
- `game/game_engine.py` — collision detection, game-over screen, difficulty settings, replay, and sound effects.
- `game/ball.py`, `game/paddle.py`, `game/brick.py` — retained existing components.
- `requirements.txt` and `README.md`.

Only **two source files need replacing**: `main.py` and `game/game_engine.py`. You do not need to edit the other three Python files.

1. Replace `main.py`

Open `main.py`, select all its contents, and replace them with this code.

```
import pygame
from game.game_engine import GameEngine


WIDTH, HEIGHT = 640, 560
FPS = 60


def main():
    pygame.init()
    screen = pygame.display.set_mode((WIDTH, HEIGHT))
    pygame.display.set_caption("Brick Breaker - Pygame Version")
    clock = pygame.time.Clock()
    engine = GameEngine(WIDTH, HEIGHT)
    running = True

    while running:
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
```

```
                running = False
                continue

            action = engine.handle_event(event)

            if action == "quit":
                running = False
            elif action == "restart":
                engine.start_new_game(engine.selected_difficulty)

        if running:
            engine.handle_input()
            engine.update()
            engine.render(screen)
            pygame.display.flip()
            clock.tick(FPS)

    engine.close()
    pygame.quit()


if __name__ == "__main__":
    main()
```

2. Replace `game/game_engine.py`

Open the `game` folder, select `game_engine.py`, and replace the **entire file** with the following code. This version integrates the four tasks.

```
import math
import array
import pygame

from .paddle import Paddle
from .ball import Ball
from .brick import Brick


WHITE = (255, 255, 255)
BG = (15, 15, 25)

BRICK_COLORS = [
    (200, 60, 60),
```

```
    (200, 140, 60),
    (200, 200, 60),
    (80, 180, 80),
    (80, 140, 200),
]

# Difficulty settings: ball speed and paddle width.
DIFFICULTIES = {
    "Easy": (4.5, 120),
    "Medium": (6.0, 100),
    "Hard": (8.0, 80),
}


class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height

        self.font = pygame.font.SysFont("Arial", 28)
        self.title_font = pygame.font.SysFont("Arial", 42, bold=True)
        self.small_font = pygame.font.SysFont("Arial", 22)

        self.selected_difficulty = "Medium"
        self._init_audio()
        self.start_new_game(self.selected_difficulty)

    def _init_audio(self):
        """Generate sound effects without requiring external audio files."""
        self.sound_enabled = False
        self.sounds = {}

        try:
            if not pygame.mixer.get_init():
                pygame.mixer.init(
                    frequency=22050,
                    size=-16,
                    channels=1,
                    buffer=512,
                )

            self.sounds = {
                "brick": self._make_tone(880, 0.07, 0.22),
                "paddle": self._make_tone(520, 0.06, 0.18),
                "wall": self._make_tone(320, 0.045, 0.14),
```

```
                "win": self._make_tone(1040, 0.24, 0.24),
                "lose": self._make_tone(180, 0.30, 0.24),
            }

            self.sound_enabled = True

        except (pygame.error, RuntimeError):
            # Continue without sound if audio is unavailable.
            self.sounds = {}
            self.sound_enabled = False

    @staticmethod
    def _make_tone(frequency, duration, volume):
        """Create a short tone using the Pygame mixer."""
        sample_rate = 22050
        sample_count = int(sample_rate * duration)
        samples = array.array("h")
        amplitude = int(32767 * volume)

        for i in range(sample_count):
            fade = min(
                1.0,
                i / 150.0,
                (sample_count - i) / 300.0,
            )

            value = int(
                amplitude
                * max(0.0, fade)
                * math.sin(
                    2.0 * math.pi * frequency * i / sample_rate
                )
            )

            samples.append(value)

        return pygame.mixer.Sound(buffer=samples.tobytes())

    def _play_sound(self, name):
        if self.sound_enabled:
            sound = self.sounds.get(name)

            if sound is not None:
                try:
                    sound.play()
```

```
                except pygame.error:
                    pass

    def start_new_game(self, difficulty="Medium"):
        """Start a fresh game with the selected difficulty."""
        if difficulty not in DIFFICULTIES:
            difficulty = "Medium"

        self.selected_difficulty = difficulty
        self.ball_speed, paddle_width = DIFFICULTIES[difficulty]

        paddle_speed = {
            "Easy": 7,
            "Medium": 7.5,
            "Hard": 8,
        }[difficulty]

        self.paddle = Paddle(
            self.width // 2 - paddle_width // 2,
            self.height - 30,
            paddle_width,
            14,
        )
        self.paddle.speed = paddle_speed

        self.ball = Ball(
            self.width // 2,
            self.height - 50,
            radius=8,
        )
        self._launch_ball()

        self.rows, self.cols = 5, 8
        self.bricks = self._build_bricks(self.rows, self.cols)

        self.lives = 3
        self.score = 0
        self.game_over = False
        self.result = None
        self._end_sound_played = False

    def _launch_ball(self):
        """Launch the ball with the speed for the selected difficulty."""
        self.ball.vx = self.ball_speed * 0.22
        self.ball.vy = -math.sqrt(
```

```
            max(1.0, self.ball_speed ** 2 - self.ball.vx ** 2)
        )

    def _build_bricks(self, rows, cols):
        bricks = []
        margin, gap, top = 30, 6, 60

        brick_w = (
            self.width - margin * 2 - gap * (cols - 1)
        ) // cols

        brick_h = 22

        for r in range(rows):
            for c in range(cols):
                x = margin + c * (brick_w + gap)
                y = top + r * (brick_h + gap)

                bricks.append(
                    Brick(x, y, brick_w, brick_h)
                )

        return bricks

    def handle_event(self, event):
        """Handle replay and exit choices on the end screen."""
        if event.type == pygame.KEYDOWN and self.game_over:
            key_to_difficulty = {
                pygame.K_1: "Easy",
                pygame.K_2: "Medium",
                pygame.K_3: "Hard",
                pygame.K_KP1: "Easy",
                pygame.K_KP2: "Medium",
                pygame.K_KP3: "Hard",
            }

            if event.key in key_to_difficulty:
                self.selected_difficulty = key_to_difficulty[event.key]
                return "restart"

            if event.key in (pygame.K_ESCAPE, pygame.K_q):
                return "quit"

        return None
```

```
    def handle_input(self):
        if self.game_over:
            return

        keys = pygame.key.get_pressed()

        if keys[pygame.K_LEFT] or keys[pygame.K_a]:
            self.paddle.move(-self.paddle.speed, self.width)

        if keys[pygame.K_RIGHT] or keys[pygame.K_d]:
            self.paddle.move(self.paddle.speed, self.width)

    def update(self):
        if self.game_over:
            return

        previous_x = self.ball.x
        previous_y = self.ball.y

        self.ball.move()

        # Left and right walls.
        if self.ball.x - self.ball.radius <= 0:
            self.ball.x = self.ball.radius

            if self.ball.vx < 0:
                self.ball.vx = abs(self.ball.vx)
                self._play_sound("wall")

        elif self.ball.x + self.ball.radius >= self.width:
            self.ball.x = self.width - self.ball.radius

            if self.ball.vx > 0:
                self.ball.vx = -abs(self.ball.vx)
                self._play_sound("wall")

        # Top wall.
        if self.ball.y - self.ball.radius <= 0:
            self.ball.y = self.ball.radius

            if self.ball.vy < 0:
                self.ball.vy = abs(self.ball.vy)
                self._play_sound("wall")

        # Paddle collision.
```

```
        paddle_rect = self.paddle.rect()

        if self.ball.rect().colliderect(paddle_rect):
            old_vx = self.ball.vx
            old_vy = self.ball.vy

            self._resolve_ball_collision(
                paddle_rect,
                previous_x,
                previous_y,
            )

            if (old_vx, old_vy) != (
                self.ball.vx,
                self.ball.vy,
            ):
                self._play_sound("paddle")

        # Brick collision: destroy at most one brick per frame.
        ball_rect = self.ball.rect()

        for brick in self.bricks:
            if brick.alive and ball_rect.colliderect(brick.rect()):
                brick_rect = brick.rect()

                brick.alive = False
                self.score += 1

                self._resolve_ball_collision(
                    brick_rect,
                    previous_x,
                    previous_y,
                )

                self._play_sound("brick")
                break

        # Lose a life when the ball passes the bottom edge.
        if self.ball.y - self.ball.radius > self.height:
            self.lives -= 1

            if self.lives <= 0:
                self._finish_game("lose")
            else:
                self._reset_ball()
```

```
        # Win after destroying every brick.
        if (
            not self.game_over
            and all(not brick.alive for brick in self.bricks)
        ):
            self._finish_game("win")

    def _resolve_ball_collision(
        self,
        target_rect,
        previous_x,
        previous_y,
    ):
        """Determine the impacted side and reflect the correct velocity."""
        radius = self.ball.radius

        previous_left = previous_x - radius
        previous_right = previous_x + radius
        previous_top = previous_y - radius
        previous_bottom = previous_y + radius

        current_left = self.ball.x - radius
        current_right = self.ball.x + radius
        current_top = self.ball.y - radius
        current_bottom = self.ball.y + radius

        # Check whether the ball crossed an edge during this frame.
        crossed_x = (
            (
                previous_right <= target_rect.left
                and current_right >= target_rect.left
            )
            or (
                previous_left >= target_rect.right
                and current_left <= target_rect.right
            )
        )

        crossed_y = (
            (
                previous_bottom <= target_rect.top
                and current_bottom >= target_rect.top
            )
            or (
```

```
                previous_top >= target_rect.bottom
                and current_top <= target_rect.bottom
            )
        )

        overlap_x = (
            min(current_right, target_rect.right)
            - max(current_left, target_rect.left)
        )

        overlap_y = (
            min(current_bottom, target_rect.bottom)
            - max(current_top, target_rect.top)
        )

        # Choose the axis of impact.
        if crossed_x and not crossed_y:
            axis = "x"
        elif crossed_y and not crossed_x:
            axis = "y"
        else:
            # For corners or ambiguous overlaps, use the smaller overlap.
            axis = "x" if overlap_x < overlap_y else "y"

        if axis == "x":
            if previous_x < target_rect.centerx:
                self.ball.x = target_rect.left - radius
                self.ball.vx = -abs(self.ball.vx)
            else:
                self.ball.x = target_rect.right + radius
                self.ball.vx = abs(self.ball.vx)

        else:
            if previous_y < target_rect.centery:
                self.ball.y = target_rect.top - radius
                self.ball.vy = -abs(self.ball.vy)
            else:
                self.ball.y = target_rect.bottom + radius
                self.ball.vy = abs(self.ball.vy)

    def _reset_ball(self):
        """Reset the ball after losing a life."""
        self.ball.x = self.width // 2
        self.ball.y = self.height - 50
        self._launch_ball()
```

```
    def _finish_game(self, result):
        """Set the result and play the ending sound exactly once."""
        if self.game_over:
            return

        self.game_over = True
        self.result = result

        if not self._end_sound_played:
            sound_name = "win" if result == "win" else "lose"
            self._play_sound(sound_name)
            self._end_sound_played = True

    def _draw_centered(self, screen, text, font, color, y):
        surface = font.render(text, True, color)
        rect = surface.get_rect(
            center=(self.width // 2, y)
        )
        screen.blit(surface, rect)

    def render(self, screen):
        screen.fill(BG)

        pygame.draw.rect(
            screen,
            WHITE,
            self.paddle.rect(),
        )

        pygame.draw.circle(
            screen,
            WHITE,
            (int(self.ball.x), int(self.ball.y)),
            self.ball.radius,
        )

        for i, brick in enumerate(self.bricks):
            if brick.alive:
                row = i // self.cols
                color = BRICK_COLORS[row % len(BRICK_COLORS)]

                pygame.draw.rect(
                    screen,
                    color,
```

```
                brick.rect(),
            )

        score_text = self.font.render(
            f"Score: {self.score}",
            True,
            WHITE,
        )

        lives_text = self.font.render(
            f"Lives: {self.lives}",
            True,
            WHITE,
        )

        screen.blit(score_text, (10, 10))
        screen.blit(
            lives_text,
            (self.width - 130, 10),
        )

        difficulty_text = self.small_font.render(
            f"Difficulty: {self.selected_difficulty}",
            True,
            WHITE,
        )

        screen.blit(
            difficulty_text,
            (10, self.height - 28),
        )

        if self.game_over:
            # Dark overlay behind the end-screen text.
            overlay = pygame.Surface(
                (self.width, self.height),
                pygame.SRCALPHA,
            )
            overlay.fill((0, 0, 0, 190))
            screen.blit(overlay, (0, 0))

            heading = (
                "YOU WIN!"
                if self.result == "win"
                else "GAME OVER"
```

```
            )

            self._draw_centered(
                screen,
                heading,
                self.title_font,
                WHITE,
                175,
            )

            self._draw_centered(
                screen,
                f"Final score: {self.score}",
                self.font,
                WHITE,
                230,
            )

            self._draw_centered(
                screen,
                "Play again? Choose a difficulty",
                self.small_font,
                WHITE,
                285,
            )

            self._draw_centered(
                screen,
                "1 - Easy     2 - Medium     3 - Hard",
                self.small_font,
                WHITE,
                330,
            )

            self._draw_centered(
                screen,
                "ESC or Q - Exit",
                self.small_font,
                WHITE,
                375,
            )

    def close(self):
        """Release audio resources when the game closes."""
        if self.sound_enabled:
```

```
            try:
                pygame.mixer.stop()
                pygame.mixer.quit()
            except pygame.error:
                pass
```

3. Run the game

From the repository root, run:

```
pip install -r requirements.txt
python -m py_compile main.py game/game_engine.py
python main.py
```

The syntax-check command should finish without errors before you launch the game.

4. Verify all four tasks

Task	What to test
1. Collision detection	Hit bricks from the left, right, top, and corners. Check that the ball changes direction appropriately.
2. Game-over screen	Lose all three lives and confirm that the end screen displays `GAME OVER` and the final score. Clear every brick to test `YOU WIN!`.
3. Replay and difficulty	On the end screen, press `1` for Easy, `2` for Medium, or `3` for Hard. Confirm that the score, lives, ball, and bricks reset.
4. Sound feedback	Listen for sounds when breaking a brick, hitting a paddle or wall, and ending the game.

The difficulty settings are:

- **Easy:** ball speed 4.5, paddle width 120 pixels.
- **Medium:** ball speed 6, paddle width 100 pixels.
- **Hard:** ball speed 8, paddle width 80 pixels.

If audio is unavailable on the computer, the game is designed to continue without sound.

Testing limitation: I checked the replacement Python files for syntax errors, but I could not launch the game in this environment because Pygame is not installed. Run the commands above and test the gameplay before submitting; I cannot guarantee every runtime behavior without that test.

5. Commit the changes

After testing, create a separate commit for each task, as required by your lab instructions. For example:

```
git add game/game_engine.py
git commit -m "Fix collision detection and bounce direction"

git add main.py game/game_engine.py
git commit -m "Add game over screen and replay difficulty"

git add game/game_engine.py
git commit -m "Add collision and game end sound effects"
```

These commands assume you commit Task 1 first, then Task 2 and Task 3 together, and finally Task 4. For the cleanest assignment history, make each task's changes and commit before starting the next task. Push only to your own repository or fork, not the original assigned repository.

If you want, I can:

- Explain how to test collision detection in detail
- Describe how to extend the game with a lives and scoring system improvement
- Explain how to add a new difficulty level (e.g., 'Insane') to the game